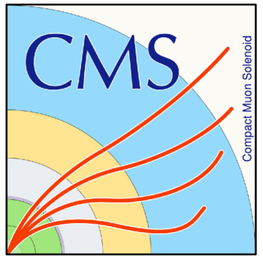
IT Unpacker Update : GPU

Tracker DPG Meeting
2026/09/02

Si Hyun Jeon, Zeynep Demiragli, Carlos Erice Cid

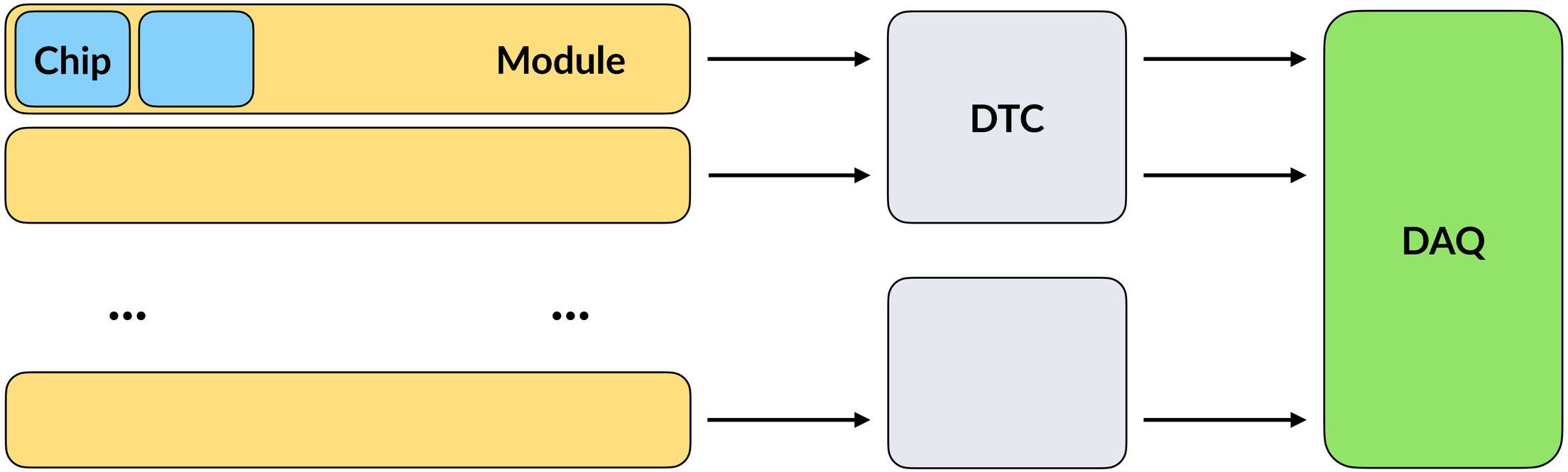


Overview

BOSTON UNIVERSITY

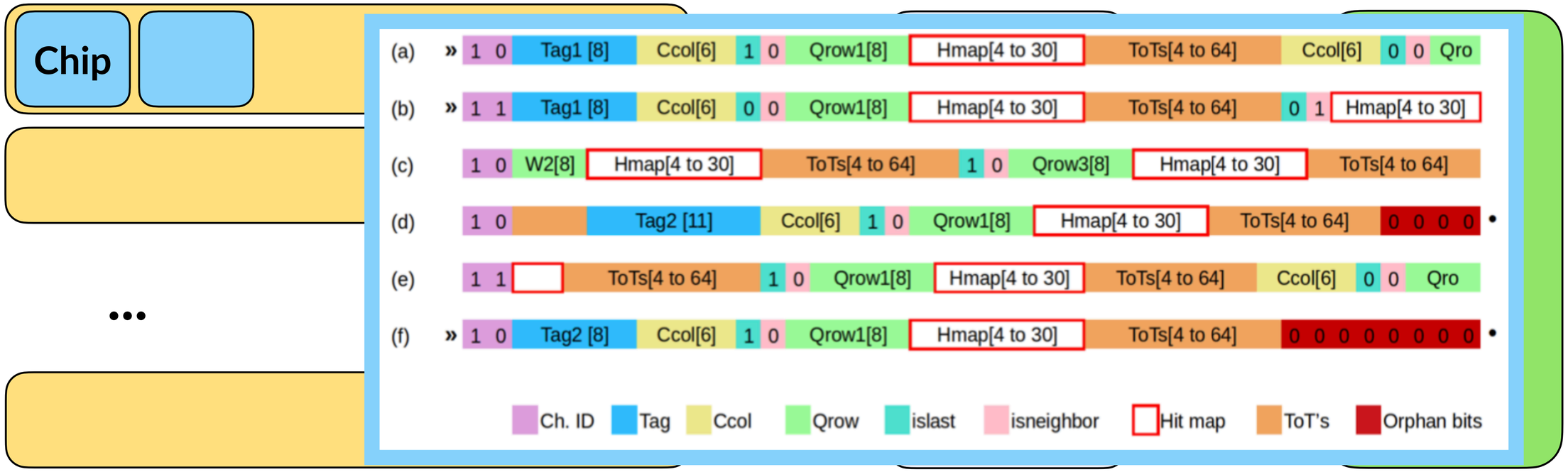
- Data flow in inner tracker system
  - RD53 chip encoding
  - DAQ data framing
- Current CMSSW implementations in CPU for the IT packer/unpacker
- Status of GPU developments in IT unpacker
  - Validation : Input and output PixelDigi checks
  - Performance : Comparison against CPU, Event and GPU block scaling tests

Data Flow in the Inner Tracker System



- Pixel hits consumed by the chips, providing us with the information row/col/adc values

Data Flow in the Inner Tracker System



- At the very first stage, chips send data in RD53 format with some level of compression (e.g. Huffman encoding of hit maps, details in the document)

Data Flow in the Inner Tracker System

- Bitstreams are then sent through the DTCs, to the DAQ
- Numbers to keep in mind : 36 DTCs, 4000 modules, and 13256 chips across the whole detector. Each DTCs have 16 s-links (~7 modules and ~23 chips per s-link, but depends on the DTC)

DAQ data format and its content discussed jointly with FW : See Sec1. in link

DAQ Data Format

Word \ Bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	S-link 128b order (following page)
0x00000000	Magic #1 (0xC3)								Ver Major (1)			Ver Minor (3)					Error Flags				Chip Count													3
0x00000002	Reserved																Trig source				BX												2
0x00000004	Orbit # MSBs																Orbit # LSBs																1
0x00000006	Reserved																																0
0x00000008	Module 1 offset (in 32-bit words)																																3
...	...																																...
...	Module M offset (in 32-bit words)																																
...	**Padding to 128 bit word size**																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 1 size (in 32-bit words)																3
...	Raw Module 1 Chip 1 binary data																																2
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 2 size (in 32-bit words)																
...	Raw Module 1 Chip 2 binary data																																
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 3 size (in 32-bit words)																...
...	Raw Module 1 Chip 3 binary data																																
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 4 size (in 32-bit words)																
...	Raw Module 1 Chip 4 binary data																																
...	...								Padding bits (End bit #)																								
...	**Padding to 128 bit word size (for Module 1)**																																0
...	...																																3
...	...																																...
...	**Padding to 128 bit word size (for Module N-1)**																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module N Chip 1 size (in 32-bit words)																3
...	Raw Module N Chip 1 binary data																																2
...	...																																...
...	...																																
...	**Padding to 128 bit word size (for Module N)**																																0
...	Magic #2 (0x3C)											Reserved																					3
...	Reserved																																2
...	Reserved																																1
END	Reserved																																0

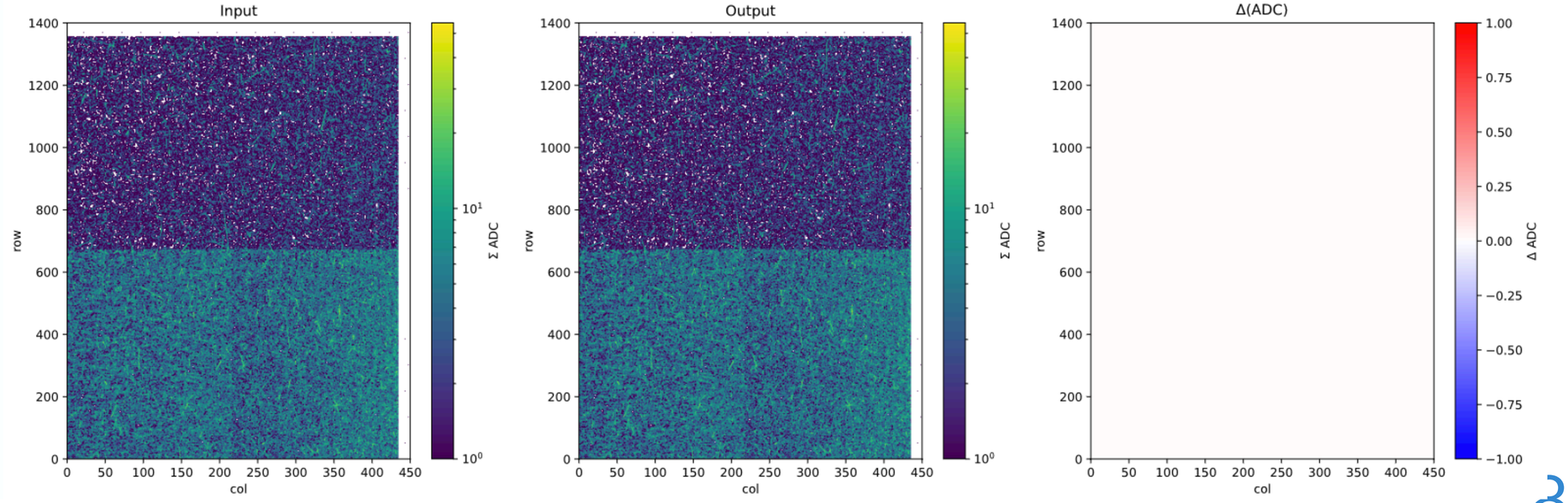
DAQ Data Format

Word \ Bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	S-link 128b order (following page)
0x00000000	Magic #1 (0xC3)								Ver Major (1)			Ver Minor (3)					Error Flags				Chip Count													3
0x00000002	Reserved																Trig source				BX												2
0x00000004	Orbit # MSBs																Orbit # LSBs																1
0x00000006	Reserved																																0
0x00000008	Module 1 offset (in 32-bit words)																																3
...	...																																...
...	Magic #3 (0xE)			Error flags				Res			End bit #						Module 1 Chip 1 size (in 32-bit words)																3
...	Raw Module 1 Chip 1 binary data																																2
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)			Error flags				Res			End bit #						Module 1 Chip 2 size (in 32-bit words)																
...	Raw Module 1 Chip 2 binary data																																
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)			Error flags				Res			End bit #						Module 1 Chip 3 size (in 32-bit words)																...
...	Raw Module 1 Chip 3 binary data																																
...	...								Padding bits (End bit #)																								
...	Magic #3 (0xE)			Error flags				Res			End bit #						Module 1 Chip 4 size (in 32-bit words)																
...	Raw Module 1 Chip 4 binary data																																
...	...								Padding bits (End bit #)																								
...	**Padding to 128 bit word size (for Module 1)**																																0
...	...																																3
...	...																																...
...	**Padding to 128 bit word size (for Module N-1)**																																0
...	Magic #3 (0xE)			Error flags				Res			End bit #						Module N Chip 1 size (in 32-bit words)																3
...	Raw Module N Chip 1 binary data																																2
...	...																																...
...	...																																
...	**Padding to 128 bit word size (for Module N)**																																0
...	Magic #2 (0x3C)								Reserved																								3
...	Reserved																																2
...	Reserved																																1
END	Reserved																																0

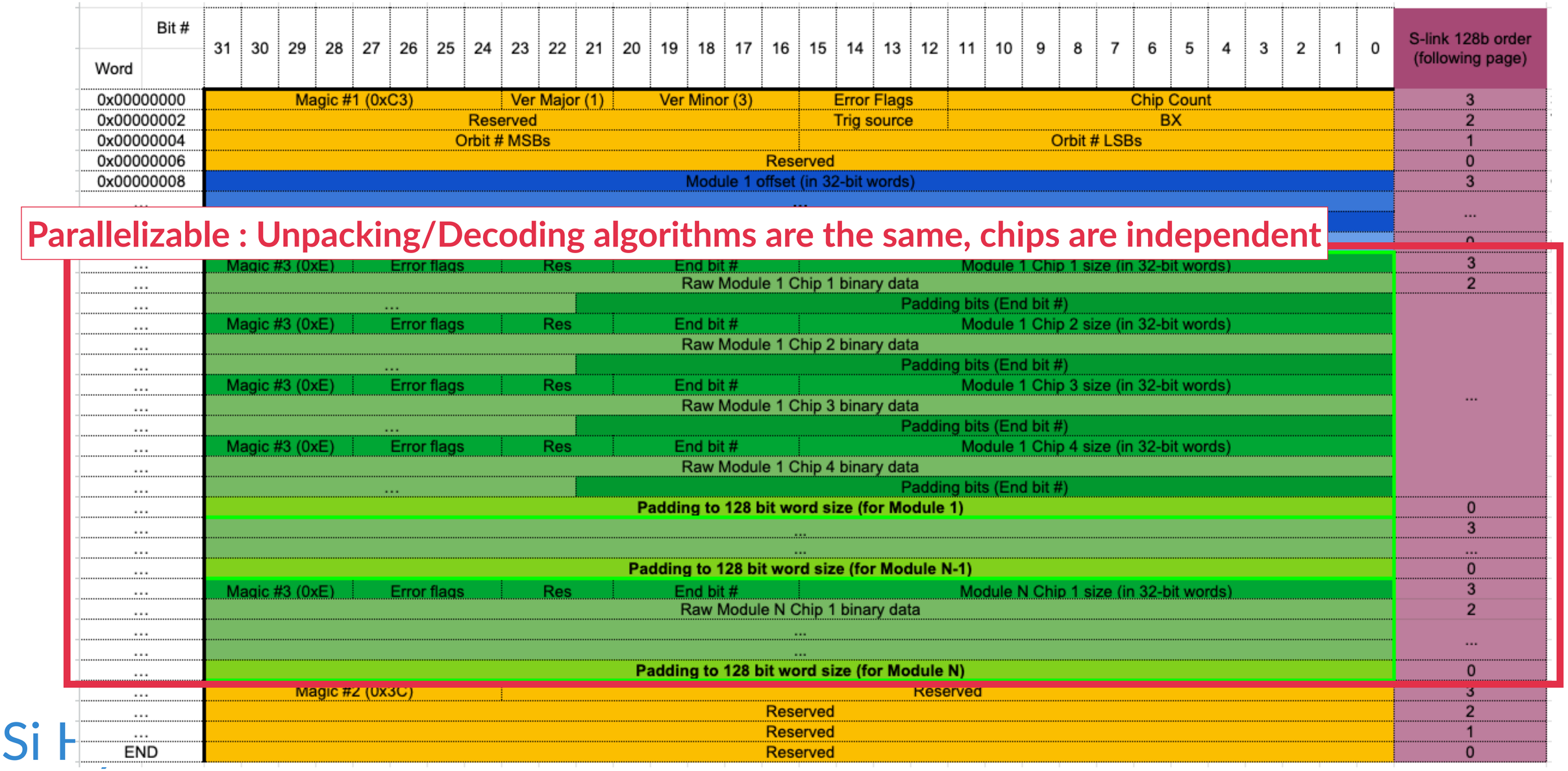
Need to unpack and decode (going back to the stage before RD53 encoding)

Status of SW Implementations

- Packer/Unpacker fully ready in P2 Tracker BES SW development area through CMSSW
  - Documented in link and presented the validation results in link
  - Current CMSSW C++ implementations able to precisely pack and unpack the to retrieve the PixelDigi records
- What next?



Next Steps?


Bit # / Word
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
S-link 128b order (following page)
0x00000000 Magic #1 (0xC3) Ver Major (1) Ver Minor (3) Error Flags Chip Count 3
0x00000002 Reserved Trig source BX 2
0x00000004 Orbit # MSBs Orbit # LSBs 1
0x00000006 Reserved 0
0x00000008 Module 1 offset (in 32-bit words) 3
Parallelizable : Unpacking/Decoding algorithms are the same, chips are independent
Magic #3 (0xE) Error flags Res End bit # Module 1 Chip 1 size (in 32-bit words) 3
Raw Module 1 Chip 1 binary data 2
Padding bits (End bit #)
Magic #3 (0xE) Error flags Res End bit # Module 1 Chip 2 size (in 32-bit words)
Raw Module 1 Chip 2 binary data
Padding bits (End bit #)
Magic #3 (0xE) Error flags Res End bit # Module 1 Chip 3 size (in 32-bit words)
Raw Module 1 Chip 3 binary data
Padding bits (End bit #)
Magic #3 (0xE) Error flags Res End bit # Module 1 Chip 4 size (in 32-bit words)
Raw Module 1 Chip 4 binary data
Padding bits (End bit #)
Padding to 128 bit word size (for Module 1) 0
3
Padding to 128 bit word size (for Module N-1) 0
Magic #3 (0xE) Error flags Res End bit # Module N Chip 1 size (in 32-bit words) 3
Raw Module N Chip 1 binary data 2
Padding to 128 bit word size (for Module N) 0
Magic #2 (0x3C) Reserved 3
Reserved 2
Reserved 1
END Reserved 0

Attempting GPU Implementations

- Wrote the unpacker part of the code to be alpaka compliant, living under link (not yet reviewed by experts)
- Started working based on CMSSW_16_0_X
- Testing on TTbar sample 200PU (produced under CMSSW_14_0_X)

Attempting GPU Implementations

- Some clarifications would help
- Output : Produces SiPixelDigisSoA (readily built)
  - clus : 0 as a placeholder
  - pdigi : Row/Col/ToT/Flag container, flag 0 as a placeholder
  - rawIdArr : Global detector element ID (detId)
  - adc : ToT value (duplicated? calibrated ToT?), kept same as pdigi.adc
  - xx/yy : Row/Col (duplicated?), kept same as pdigi.row/pdigi.col
  - moduleId : Module index from 0-3999

Unpacking/Decoding Logic with GPUs

- H2D of whole 576 RawDataBuffer (16 s-links per DTC, 36 DTCs), flattened/jagged array

First parse out the headers →

Word \ Bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	S-link 128b order (following page)
0x00000000	Magic #1 (0xC3)								Ver Major (1)			Ver Minor (3)					Error Flags				Chip Count													3
0x00000002	Reserved																Trig source				BX												2
0x00000004	Orbit # MSBs																Orbit # LSBs																1
0x00000006	Reserved																																0
0x00000008	Module 1 offset (in 32-bit words)																																3
...	...																																
...	Module M offset (in 32-bit words)																																...
...	**Padding to 128 bit word size**																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #																						3
...	Raw Module 1 Chip 1 binary data																																2
...	...											Padding bits (End bit #)																					
...	Magic #3 (0xE)				Error flags				Res			End bit #																						
...	Raw Module 1 Chip 2 binary data																																
...	...											Padding bits (End bit #)																					
...	Magic #3 (0xE)				Error flags				Res			End bit #																						
...	Raw Module 1 Chip 3 binary data																																
...	...											Padding bits (End bit #)																					
...	Magic #3 (0xE)				Error flags				Res			End bit #																						
...	Raw Module 1 Chip 4 binary data																																
...	...											Padding bits (End bit #)																					
...	**Padding to 128 bit word size (for Module 1)**																																0
...	...																																3
...	...																																...
...	**Padding to 128 bit word size (for Module N-1)**																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #																						3
...	Raw Module N Chip 1 binary data																																2
...	...																																...
...	...																																
...	**Padding to 128 bit word size (for Module N)**																																0
...	Magic #2 (0x3C)																	Reserved																3
...	Reserved																																2
...	Reserved																																1
END	Reserved																																0

Unpacking/Decoding Logic with GPUs

- H2D of whole 576 RawDataBuffer (16 s-links per DTC, 36 DTCs), flattened/jagged array

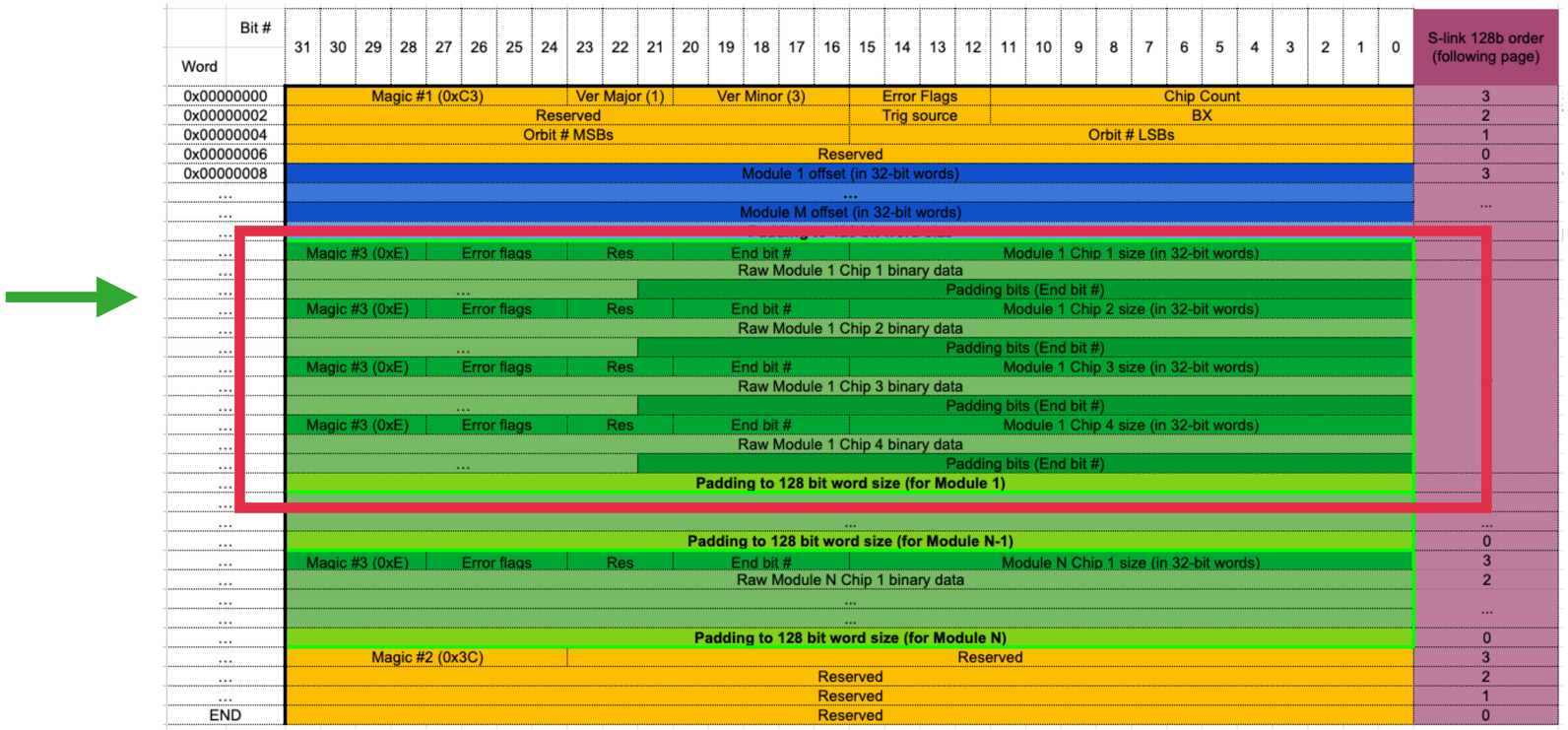
Read the module offsets →

Word \ Bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	S-link 128b order (following page)
0x00000000	Magic #1 (0xC3)								Ver Major (1)			Ver Minor (3)					Error Flags				Chip Count													3
0x00000002	Reserved																Trig source				BX												2
0x00000004	Orbit # MSBs																Orbit # LSBs																1
0x00000006	Reserved																																0
0x00000008	Module 1 offset (in 32-bit words)																																3
…	…																																…
…	Module M offset (in 32-bit words)																																
…	Padding to 128 bit word size																																0
…	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 1 size (in 32-bit words)																3
…	Raw Module 1 Chip 1 binary data																																2
…	…											Padding bits (End bit #)																					
…	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 2 size (in 32-bit words)																
…	Raw Module 1 Chip 2 binary data																																
…	…											Padding bits (End bit #)																					
…	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 3 size (in 32-bit words)																…
…	Raw Module 1 Chip 3 binary data																																
…	…											Padding bits (End bit #)																					
…	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 4 size (in 32-bit words)																
…	Raw Module 1 Chip 4 binary data																																
…	…											Padding bits (End bit #)																					
…	Padding to 128 bit word size (for Module 1)																																0
…	…																																3
…	…																																…
…	Padding to 128 bit word size (for Module N-1)																																0
…	Magic #3 (0xE)				Error flags				Res			End bit #					Module N Chip 1 size (in 32-bit words)																3
…	Raw Module N Chip 1 binary data																																2
…	…																																…
…	…																																
…	Padding to 128 bit word size (for Module N)																																0
…	Magic #2 (0x3C)																	Reserved															3
…	Reserved																																2
…	Reserved																																1
END	Reserved																																0

Unpacking/Decoding Logic with GPUs

- H2D of whole 576 RawDataBuffer (16 s-links per DTC, 36 DTCs), flattened/jagged array

From the read offsets, parse out modules and create module data blocks



Unpacking/Decoding Logic with GPUs

- H2D of whole 576 RawDataBuffer (16 s-links per DTC, 36 DTCs), flattened/jagged array

From encoded chip sizes in the headers, parse out chips and create chip data blocks

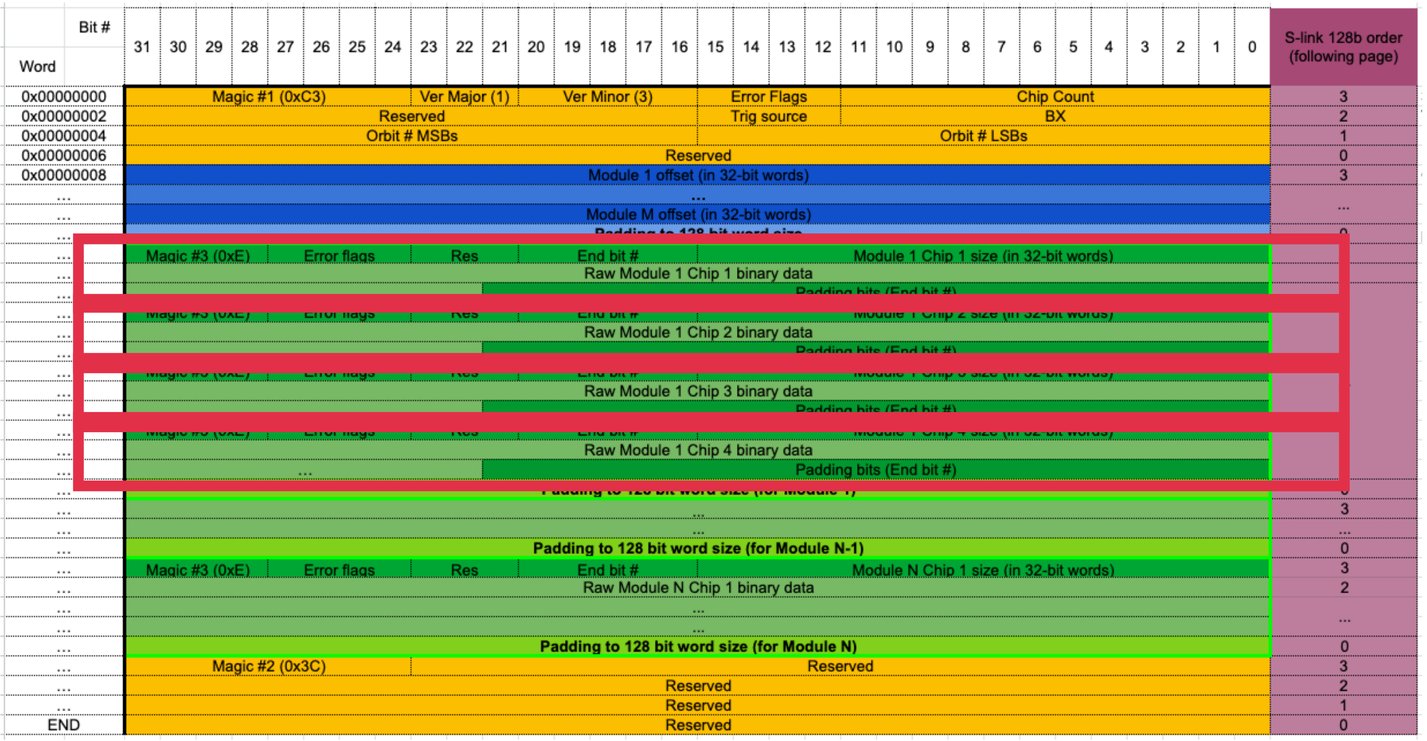
Word \ Bit #	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	S-link 128b order (following page)
0x00000000	Magic #1 (0xC3)								Ver Major (1)			Ver Minor (3)					Error Flags				Chip Count													3
0x00000002	Reserved																Trig source				BX												2
0x00000004	Orbit # MSBs																Orbit # LSBs																1
0x00000006	Reserved																																0
0x00000008	Module 1 offset (in 32-bit words)																																3
...	...																																
...	Module M offset (in 32-bit words)																																...
...	Padding to 128 bit word size																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 1 size (in 32-bit words)																
...	Raw Module 1 Chip 1 binary data																																
...	...																Padding bits (End bit #)																
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 2 size (in 32-bit words)																
...	Raw Module 1 Chip 2 binary data																																
...	...																Padding bits (End bit #)																
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 3 size (in 32-bit words)																...
...	Raw Module 1 Chip 3 binary data																																
...	...																Padding bits (End bit #)																
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module 1 Chip 4 size (in 32-bit words)																
...	Raw Module 1 Chip 4 binary data																																
...	...																Padding bits (End bit #)																
...	Padding to 128 bit word size (for Module 1)																																0
...	...																																3
...	...																																...
...	Padding to 128 bit word size (for Module N-1)																																0
...	Magic #3 (0xE)				Error flags				Res			End bit #					Module N Chip 1 size (in 32-bit words)																3
...	Raw Module N Chip 1 binary data																																2
...	...																																...
...	...																																
...	Padding to 128 bit word size (for Module N)																																0
...	Magic #2 (0x3C)																Reserved																3
...	Reserved																																2
...	Reserved																																1
END	Reserved																																0

Unpacking/Decoding Logic with GPUs

- H2D of whole 576 RawDataBuffer (16 s-links per DTC, 36 DTCs), flattened/jagged array

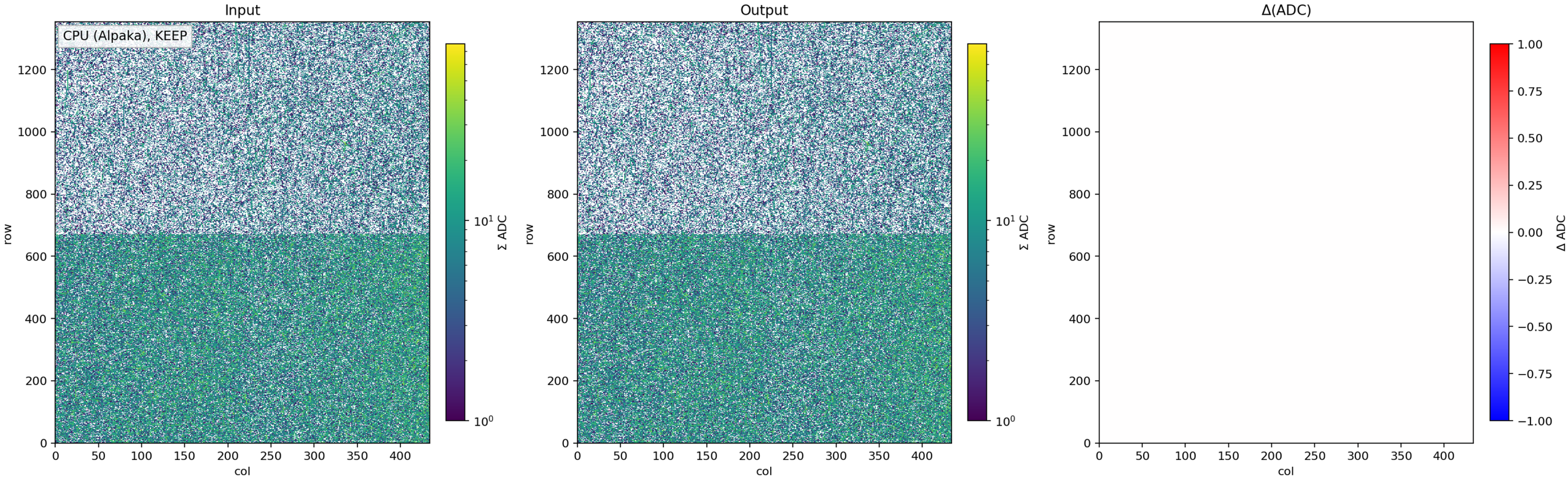
So now we have chip data blocks individually separated

Parallelized decoding of chip data blocks to retrieve individual pdigi row/col/adc



Validation

- Recovery test : (row/col/adc) agree between input (before packing) and output (after packing/unpacking roundtrip)

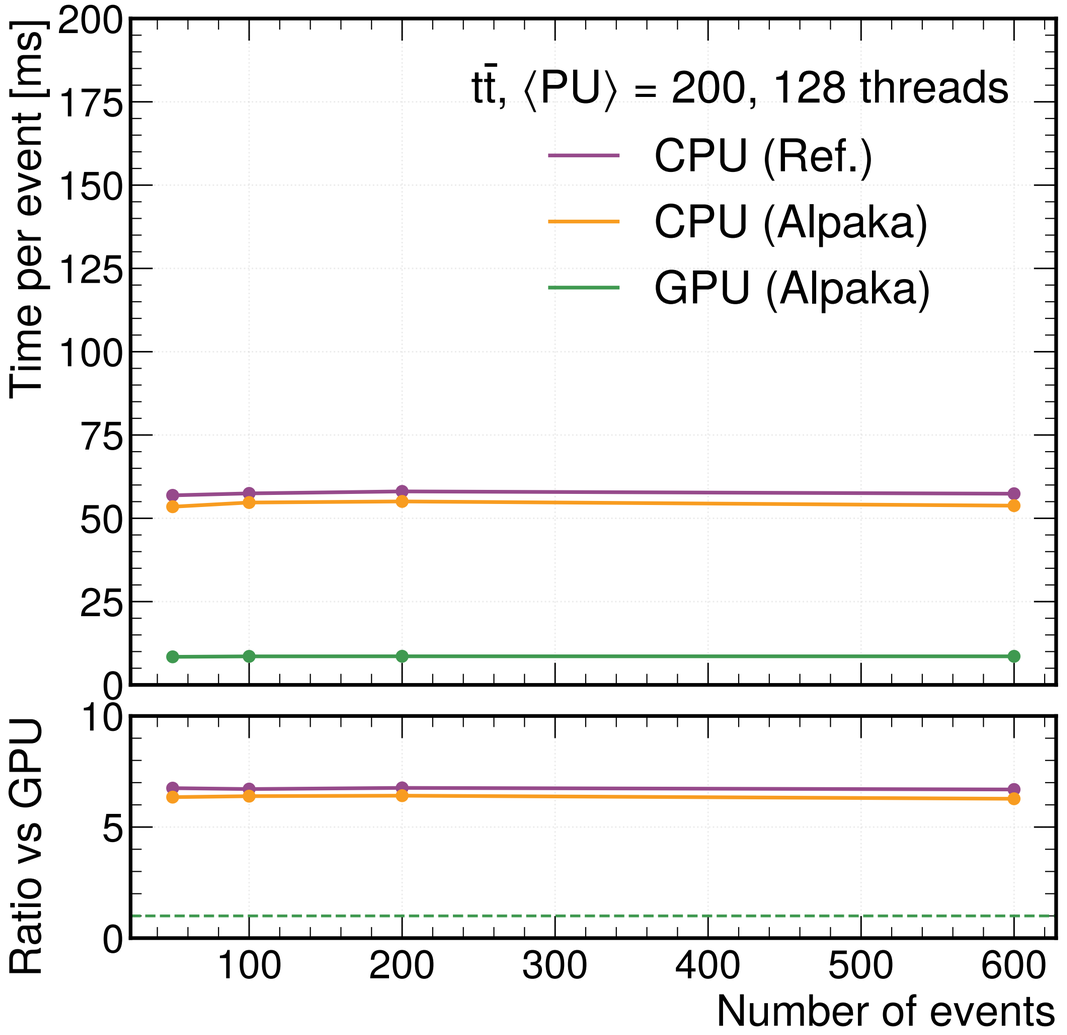


Performance of Unpacking with GPUs

- Factor of ~7 gain in time per event comparing CPU (Ref.) and GPU (Alpaka)
- CPU (Alpaka) slightly faster than CPU (Ref.) most likely due to the flushing outputs : DetSetVector gets created at the last stage for CPU (Ref.)

CPU (Ref.)	CPU (Alpaka)	GPU (Alpaka)
57.4 ms	53.8 ms	8.58 ms

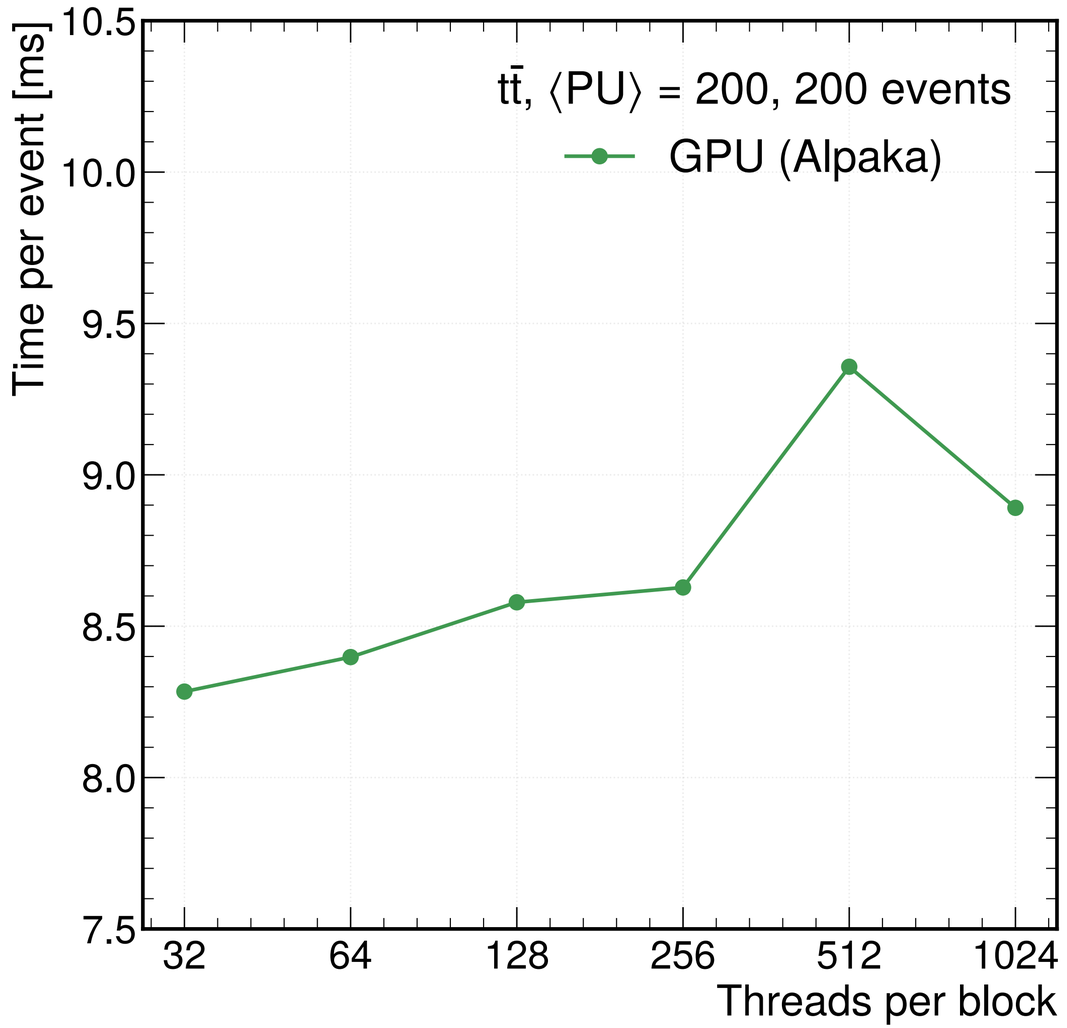
Ran in lxplus-gpu.cern.ch with Tesla T4 GPU



Performance of Unpacking with GPUs

- O(13K) chips are parallelized, scanning 32-1024 threads per GPU block
- Entire spread is within 12% (from 8.3 to 9.4 ms)
- Bottleneck from chips with longer bitstreams with more hits
- Unpacking with GPUs help with H2D time : 1.044 MB (per event) before unpacking becomes larger, 10.10 MB after decoding

Ran in lxplus-gpu.cern.ch with Tesla T4 GPU



Summary

- Worked on GPU implementations of IT packer/unpacker
- Results validated : Round trip test successful, input and output pixel digis match
- Checked the performance : Roughly ~7 times faster with GPU
- Would like to ask…
  - Is my understanding on SiPixelDigisSoA columns correct?
  - Is there a dedicated GPU machine to do the performance checks or lxplus-gpu good enough?